1 Estimation of the Disease Progression Model

Para el análisis de datos con estructura espacial con el objetivo de estimar el avance de enfermedades, así como para el estudio de datos longitudinales se desarrollaron las siguientes herramientas de software: Leasp para C++ y Leaspy para Python. El segundo es utilizado mayoritariamente para estimar la progresión de enfermedades a largo plazo, imputar valores faltantes y la simulación de cohortes virtuales.

1.1 Leasp: Paquete de C++ para el Análisis de Datos Longitudinales Estructurados Espacialmente

Este paquete trabaja con datos obtenidos tras la observación de pacientes a lo largo del tiempo. Su función se basa en: reconstruir la trayectoria a largo plazo de los pacientes a partir de las observaciones, ubicar cada medida de cada paciente en relación con la linea de tiempo promedio grupal y medir el efecto de ciertos cofactores como por ejemplo el género en el transcurso de la señal.

Esta herramienta puede trabajar tanto con variables escalares, como puntuaciones clínicas o evaluaciones cognitivas; como con datos de red (PET SUVR).

El diseño de Leasp se centra en la interacción entre un algoritmo y un modelo basado en entradas escalares y de red. Con esta biblioteca se tiene acceso a distintas carpetas que permite el uso de varias funcionalidades:

- Aplicación de procedimientos matemáticos
  - Calibración
  - Personalización
  - Simulación
- Funciones de procesamiento
- Herramientas de visualización

Para su correcta utilización a continuación se muestran algunas instrucciones y pasos que se deben seguir:

Requisitos. Es necesario instalar las siguientes bibliotecas: *tinyxml* (lectura y preprocesamiento de archivos xml), *googletest* (pruebas de unidad y funcional) y *Armadillo* (uso de algebra lineal para vectores y matrices)

Instalación. Desde la terminal, ejecutar los comandos que se muestran a continuación:

1. `git clone https://gitlab.com/icm-institute/aramislab/leasp`
2. `git submodule init`
3. `git submodule update`
4. `. mkdir build && cd build`
5. `. cmake --options`
6. `. make --options}`

Ejecución del software. Agregación del software a `/protect/T1/textdollarPATH` y ejecucion del siguiente comando:

```
leasp fit model_stgs.xml algorithm_stgs.xml data_stgs.xml
                    sampler_stgs.xml
```

- `fit`: Ajuste de la curva promedia grupal a medidas individuales.
- `model_settings`: Da información de los parámetros y del tipo de modelo que se va a ejecutar.
- `algorithm_settings.xml`: Define parámetros del algoritmo como el numero total de iteraciones.
- `data_settings.xml`: Recopila la ruta de los archivos de datos.
- `sampler_settings.xml`: Enumera las muestras utilizadas en el procedimiento de estimación, junto con sus parámetros.

Salidas de la estimación. Se producen tres archivos en el proceso: `convergence_parameters.csv` muestra la convergencia del modelo, `population_parameters.csv` define parametros para el promedio grupal a largo plazo, y `individual_parameters.csv` que son parametros temporales y espaciales para la trayectoria individual.

1.2 Leaspy: Toolbox de Python para Aprender Patrones Espaciotemporales de la Progresión de la Enfermedad

Para el estudio de datos longitudinales y ver la progresión de enfermedades a largo plazo en Python se desarrollo la herramienta Leaspy. Basándose en el modelo de efectos mixtos, permite la reconstrucción de la evolución promedia e individual. Además, permite abordar problemas de los estudios de datos longitudinales, como el que cada paciente tenga un numero de observaciones distinto, permite trabajar con valores faltantes y medidas de datos estructurados espacialmente (imágenes o redes), gracias a la geometría de Riemann. También es de gran utilidad trabajando con medidas en un periodo de tiempo corto, cuando el fenómeno de estudio es de larga duración. Otras aplicaciones permiten predecir puntos temporales futuros y simular nuevas cohortes a partir de los parámetros obtenidos de un ya existente. La calidad del paquete se garantiza mediante pruebas unitarias, control de versiones git e integración continua.

El siguiente modelo, basado en efectos mixtos, representa un conjunto de datos longitudinales $y = (y_{ij}, t_{ij})$ $1 \leq i \leq n, 1 \leq j \leq k_i$ de n pacientes. El numero de observaciones de cada individuo i se representa con k_i, siendo los tiempos de cada observación $t_{i1} < ... < t_{ik_i}$.

$$y_{ij} = f(\theta_{geom}, z_i, t_{ij}) + \epsilon_{ij}$$

θ_{geom} son los parámetros del modelo, z_i es la variación de cada individuo respecto a la trayectoria promedio grupal y ϵ_{ij} es un ruido gaussiano. El objetivo es calcular los parámetros anteriores que hagan que los datos sean lo más verosímiles posibles. Con Leaspy se pueden realizar los siguientes procedimientos:

1. Calibración: consiste en estimar y a partir de θ, obteniendo la curva promedio del grupo a largo plazo. Esta basada en el algoritmo MCMC-SAEM y la varianza adaptativa y el descenso de gradiente mejoran la velocidad de convergencia.

2. Personalización: Teniendo $\hat{\theta}$, se reconstruyen las trayectorias individuales obteniendo la variable oculta z_i^*. Este procedimiento estadístico incluye métodos Powell y L-BFGS.

3. Reconstrucción, imputación de valores faltantes y predicción: Con las variables resultantes de los dos pasos anteriores, se calcula $y_{ij}^* = f(\hat{\theta}, z_i^*, t_{ij})$ para distintos periodos de tiempo.

4. Simulación: Se genera un individuo completamente nuevo (z_i^0) a partir de $\hat{\theta}$ y un conjunto de variables individuales, pudiendo repetir el proceso hasta conseguir una cohorte virtual. Este método está basado en estimaciones condicionales gaussianas multivariadas y en funciones de densidad kernel.

La programación en Python de los cuatro procesos anteriores sigue una metodología bastante amigable para le usuario, ya que menos para la reconstrucción, el resto se estructuran de la siguiente manera:

- **Definición de parámetros:** el objeto `AlgorithmSettings` se encarga de esta función. En su llamada se fija el algoritmo que se quiere utilizar y otras variables. También se pueden cargar a través de kwargs o archivos json.

- **Llamada a las funciones:** tras el paso anterior, se realiza la llamada al método que se este llevando a cabo, introduciendo los parámetros definidos: `leaspy.fit(...)` para la calibración, `leaspy.personalize(...)` para personalizar el modelo, `leaspy.reconstruct(...)` para reconstruirlo y `leaspy.simulate(...)`para realizar una simulación.

- **Visualización y registro:** Leaspy también permite guardar archivos de registro y otras variables en archivos csv y json, lo que también permite volver a cargarlos en el programa; y se pueden visualizar perfiles de progresión o reconstrucciones de trayectorias.

Con todas estas herramientas, Leaspy es un entorno versátil y de fácil acceso que permite introducirse en el mundo del análisis y la interpretación de datos longitudinales a través de Python. Permite abordar tareas estadísticas gracias a los modelos y algoritmos en los que esta basado , pudiendo estimara parámetros y simular datos. Además es accesibles desde distintos sistemas operativos como Windows o Linux.